



ML Packages

Julius Adebayo, Lawrence Wong, Shashank Srikant



Overview

1. SciKit Learn
2. PyTorch
3. TensorFlow

All code demonstrated here is available on:

<https://colab.research.google.com/drive/1v6BWB7n7BMnNgGylbF0iQhC-eC8rLefB?authuser=1#scrollTo=APkoLQkU3vbB>

matplotlib

NumPy



seaborn

pandas



Data Visualization

```
>> df.info()
```

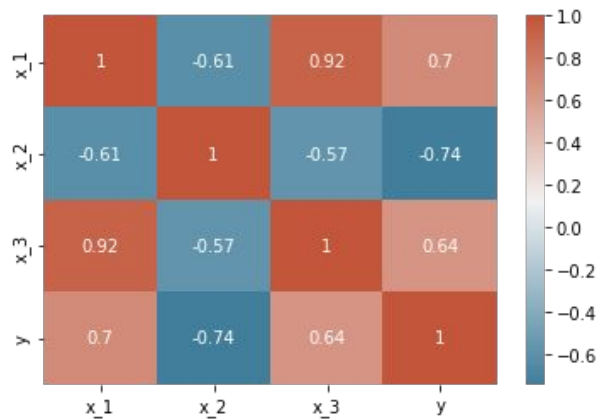
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 506 entries, 0 to 505
Data columns (total 4 columns):
 #   Column  Non-Null Count  Dtype  
---  -
 0    x_1     506 non-null    float64
 1    x_2     506 non-null    float64
 2    x_3     506 non-null    float64
 3     y      506 non-null    float64
dtypes: float64(4)
memory usage: 15.9 KB
```

```
>> df.head()
```

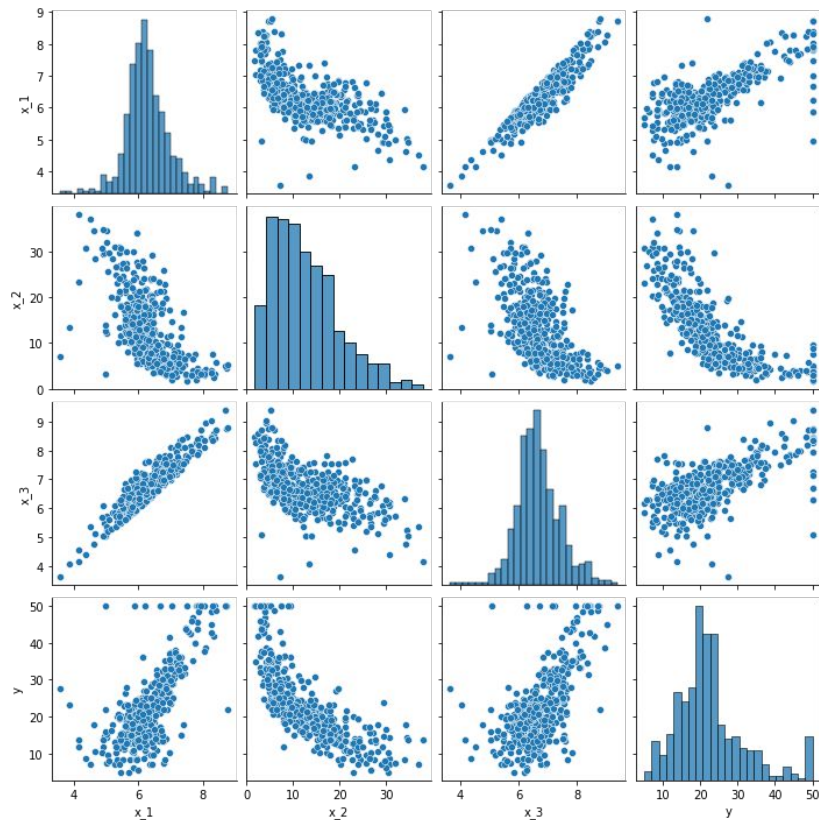
	x_1	x_2	x_3	y
0	6.575	4.98	7.457026	24.0
1	6.421	9.14	6.621079	21.6
2	7.185	4.03	7.674369	34.7
3	6.998	2.94	8.118447	33.4
4	7.147	5.33	8.080779	36.2

Data Visualization

```
>> correlation_matrix = df.corr().round(2)
>> cmap = sns.diverging_palette(230, 20, as_cmap=True)
>> sns.heatmap(data=correlation_matrix, annot=True,
               cmap=cmap)
```



```
>> sns.pairplot(df)
```





Data Pre-Processing

```
>> X, y = df[['x_1', 'x_2', 'x_3']], df[['y']]
>> X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=1)
>> X_train.shape, X_test.shape, y_train.shape, y_test.shape
((404, 3), (102, 3), (404, 1), (102, 1))
```

```
# Fit and transform train split.
```

```
>> X_train = X_train.drop(['x_3'], axis=1)
>> X_train['x_2'] = np.log(X_train['x_2'])
>> mms = MinMaxScaler()
>> X_train = mms.fit_transform(X_train)
```

```
# Transform test split.
```

```
>> X_test = X_test.drop(['x_3'], axis=1)
>> X_test['x_2'] = np.log(X_test['x_2'])
>> X_test = mms.transform(X_test)
```



Ridge Regression

```
>> model = Ridge(alpha=1.0, fit_intercept=True)
>> model.fit(X_train, y_train)
>> scoring = make_scorer(metrics.mean_squared_error, greater_is_better=False)
>> scoring(model, X_test, y_test) # calls model.predict()
-26.074
```

$$J_{n,\lambda}(\theta) = \frac{1}{2} \sum_{t=1}^n (y^{(t)} - \theta^\top x^{(t)})^2 + \frac{\lambda}{2} \sum_i^d \theta_i^2.$$



Custom Ridge Regression (.__init__)

```
class CustomRidge(BaseEstimator, RegressorMixin):  
    def __init__(self,  
                 loss_function = metrics.mean_squared_error,  
                 alpha: float = 1.0,  
                 fit_intercept: bool = True):  
  
        self.loss_function = loss_function  
        self.alpha = alpha  
        self.fit_intercept = fit_intercept  
        self.theta = None  
  
        self._param_names = ['loss_function', 'alpha', 'fit_intercept']
```


Custom Ridge Regression (fit)

```
def fit(self, X=None, y=None):  
    if self.theta is None:  
        if self.fit_intercept:  
            num_coefs = X.shape[-1] + 1  
        else:  
            X.shape[-1]  
        self.theta = np.ones(num_coefs)  
  
    result = minimize(self.objective_function,  
                      self.theta,  
                      args=(X, y),  
                      method='BFGS',  
                      options={'maxiter': 500})  
    self.theta = result.x
```

```
def objective_function(self, theta, X, y):  
    if self.fit_intercept:  
        y_hat = theta[0] + X@theta[1:]  
        loss = self.loss_function(y, y_hat)  
        reg = self.alpha *  
            np.linalg.norm(theta[1:])**2  
    else:  
        y_hat = X@theta  
        loss = self.loss_function(y, y_hat)  
        reg = self.alpha *  
            np.linalg.norm(theta)**2  
    return loss + reg
```



Custom Ridge Regression (predict)

```
def predict(self, X):  
    if self.fit_intercept:  
        return self.theta[0] + X@self.theta[1:]  
    else:  
        return X@self.theta
```

```
>> model = CustomRidge(metrics.mean_absolute_error, alpha=1.0, fit_intercept=True)
```

```
>> model.fit(X_train, y_train)
```

```
>> scoring(model, X_test, y_test)
```

```
-33.535
```

```
>> model = CustomRidge(metrics.mean_squared_error, alpha=1.0, fit_intercept=True)
```

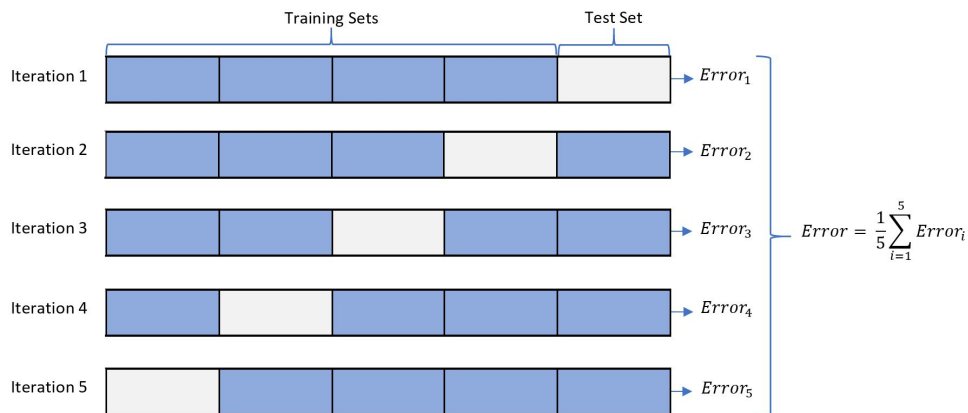
```
>> model.fit(X_train, y_train)
```

```
>> scoring(model, X_test, y_test)
```

```
-25.579
```

Cross Validation

```
>> model = CustomRidge(lf2, alpha=1.0, fit_intercept=True)
>> cross_val_score(model, X_train, y_train, cv=5, scoring=scoring)
array([-23.46017923, -30.58132272, -19.29754502, -25.63186516, -26.13599617])
```





Pipelines

- Pipeline
- ColumnTransformer

```
>> x_1_pipeline = Pipeline([  
    ("MinMaxScaler#1", MinMaxScaler())])
```

```
>> x_2_pipeline = Pipeline([  
    ("LogTransformer#1", FunctionTransformer(np.log)),  
    ("MinMaxScaler#2", MinMaxScaler())])
```

```
>> featurization = ColumnTransformer(transformers=[  
    ("x_1_pipeline", x_1_pipeline, [0]),  
    ('x_2_pipeline', x_2_pipeline, [1])])
```

```
>> pipeline = Pipeline([  
    ('features', featurization),  
    ('model', CustomRidge())])
```

```
>> pipeline.fit(X_train, y_train)  
>> scoring(pipeline, X_test, y_test)  
-25.579
```



GridSearchCV

```
>> param_grid = {
    'model__alpha': [0, 1, 10, 100],
    'model__fit_intercept': [True, False],
    'model__loss_function': [lf1, lf2]}

>> scoring = make_scorer(metrics.mean_squared_error, greater_is_better=False)
>> search = GridSearchCV(estimator=pipeline, param_grid=param_grid, n_jobs=-1, cv=5, scoring=scoring)
>> search.fit(X_train, y_train)

>> search.best_score_
-25.0057

>> search.best_params_
{'model__alpha': 0, 'model__fit_intercept': True, 'model__loss_function': 'mean_squared_error'}

>> scoring(search.best_estimator_, X_test, y_test)
-25.2903
```



https://github.com/pytorch



pytorch

📍 where the eigens are valued

🔗 <https://pytorch.org>

Verified

🏠 Overview

💻 Repositories 60

📦 Packages

👤 People 51

📁 Projects 2

Pinned



pytorch

Public

Tensors and Dynamic neural networks in Python wi



C++



54.1k



15k



examples

Public

A set of examples around pytorch in Vision, Text, R



Python



17.8k



8.4k

People



```
import torch
```

Tensors, DataLoaders

CPU vs. GPU

**Autograd,
Computation graphs**

torch.Module(NN)


```
import torch
```

Tensors, DataLoaders

CPU vs. GPU

Autograd,
Computation graphs

torch.Module(NN)

Pytorch Tensors

```
print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)
```

```
print(type(X_train), type(X_test), type(y_train), type(y_test))
```

```
>> (404, 2) (102, 2) (404, 1) (102, 1)
```

```
<class 'numpy.ndarray'> <class 'numpy.ndarray'> <class 'numpy.ndarray'> <class  
'numpy.ndarray'>
```

Pytorch Tensors

`import torch`

```
print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)
```

```
print(type(X_train), type(X_test), type(y_train), type(y_test))
```

```
>> (404, 2) (102, 2) (404, 1) (102, 1)
```

```
<class 'numpy.ndarray'> <class 'numpy.ndarray'> <class 'numpy.ndarray'> <class  
'numpy.ndarray'>
```

```
X_train, X_test = torch.tensor(X_train), torch.tensor(X_test)
```

```
y_train, y_test = torch.tensor(y_train), torch.tensor(y_test)
```

Pytorch Tensors

```
print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)
print(type(X_train), type(X_test), type(y_train), type(y_test))
```

```
>> (404, 2) (102, 2) (404, 1) (102, 1)
<class 'numpy.ndarray'> <class 'numpy.ndarray'> <class 'numpy.ndarray'> <class
'numpy.ndarray'>
```

```
X_train, X_test = torch.tensor(X_train), torch.tensor(X_test)
y_train, y_test = torch.tensor(y_train), torch.tensor(y_test)
```

```
print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)
print(type(X_train), type(X_test), type(y_train), type(y_test))
```

```
>> torch.Size([404, 2]) torch.Size([102, 2]) torch.Size([404, 1]) torch.Size([102, 1])
<class 'torch.Tensor'> <class 'torch.Tensor'> <class 'torch.Tensor'> <class 'torch.Tensor'>
```

Pytorch Tensors

```
print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)
print(type(X_train), type(X_test), type(y_train), type(y_test))
```

```
>> (404, 2) (102, 2) (404, 1) (102, 1)
<class 'numpy.ndarray'> <class 'numpy.ndarray'> <class 'numpy.ndarray'> <class
'numpy.ndarray'>
```

```
X_train, X_test = torch.tensor(X_train, dtype=torch.float), torch.tensor(X_test, dtype=torch.float)
y_train, y_test = torch.tensor(y_train, dtype=torch.float), torch.tensor(y_test, dtype=torch.float)
```

Pytorch Tensors

```
print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)
print(type(X_train), type(X_test), type(y_train), type(y_test))
```

```
>> (404, 2) (102, 2) (404, 1) (102, 1)
<class 'numpy.ndarray'> <class 'numpy.ndarray'> <class 'numpy.ndarray'> <class
'numpy.ndarray'>
```

```
X_train, X_test = torch.tensor(X_train, dtype=torch.float), torch.tensor(X_test, dtype=torch.float)
y_train, y_test = torch.tensor(y_train, dtype=torch.float), torch.tensor(y_test, dtype=torch.float)
```

```
print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)
print(type(X_train), type(X_test), type(y_train), type(y_test))
```

```
>> torch.Size([404, 2]) torch.Size([102, 2]) torch.Size([404, 1]) torch.Size([102, 1])
<class 'torch.Tensor'> <class 'torch.Tensor'> <class 'torch.Tensor'> <class 'torch.Tensor'>
```

DataLoaders

```
from torch.utils.data import Dataset

class MyDataset(Dataset):
    def __init__(self, Xs, ys):
        self.Xs = Xs
        self.ys = ys

    def __len__(self):
        'Returns the total number of samples'
        return len(self.Xs)

    def __getitem__(self, index):
        'Generates one sample of data'
        X = self.Xs[index]
        y = self.ys[index]

        # Can use this method to load data from files using index
        # X = torch.load('data/' + index + '.pt')
        return X, y
```

DataLoaders

```
print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)
```

```
print(type(X_train), type(X_test), type(y_train), type(y_test))
```

```
>> torch.Size([404, 2]) torch.Size([102, 2]) torch.Size([404, 1]) torch.Size([102, 1])  
<class 'torch.Tensor'> <class 'torch.Tensor'> <class 'torch.Tensor'> <class 'torch.Tensor'>
```


DataLoaders

```
import torch.utils.data.DataLoader
```

```
print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)
```

```
print(type(X_train), type(X_test), type(y_train), type(y_test))
```

```
>> torch.Size([404, 2]) torch.Size([102, 2]) torch.Size([404, 1]) torch.Size([102, 1])  
<class 'torch.Tensor'> <class 'torch.Tensor'> <class 'torch.Tensor'> <class 'torch.Tensor'>
```

```
train_dataset = MyDataset(X_train, y_train)
```

```
test_dataset  = MyDataset(X_test, y_test)
```

```
train_data_iterator = DataLoader(train_dataset, shuffle=True, batch_size=16)
```

```
test_data_iterator  = DataLoader(test_dataset, shuffle=True, batch_size=16)
```

DataLoaders

```
print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)
```

```
print(type(X_train), type(X_test), type(y_train), type(y_test))
```

```
>> torch.Size([404, 2]) torch.Size([102, 2]) torch.Size([404, 1]) torch.Size([102, 1])  
<class 'torch.Tensor'> <class 'torch.Tensor'> <class 'torch.Tensor'> <class 'torch.Tensor'>
```

```
train_dataset = MyDataset(X_train, y_train)
```

```
test_dataset  = MyDataset(X_test, y_test)
```

```
train_data_iterator = DataLoader(train_dataset, shuffle=True, batch_size=16)
```

```
test_data_iterator  = DataLoader(test_dataset, shuffle=True, batch_size=16)
```

```
print(len(train_data_iterator))
```

```
>> ?
```

DataLoaders

```
print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)
```

```
print(type(X_train), type(X_test), type(y_train), type(y_test))
```

```
>> torch.Size([404, 2]) torch.Size([102, 2]) torch.Size([404, 1]) torch.Size([102, 1])  
<class 'torch.Tensor'> <class 'torch.Tensor'> <class 'torch.Tensor'> <class 'torch.Tensor'>
```

```
train_dataset = MyDataset(X_train, y_train)
```

```
test_dataset  = MyDataset(X_test, y_test)
```

```
train_data_iterator = DataLoader(train_dataset, shuffle=True, batch_size=16)
```

```
test_data_iterator  = DataLoader(test_dataset, shuffle=True, batch_size=16)
```

```
print(len(train_data_iterator))
```

```
>> 26
```

(404/16 = 25.25)

DataLoaders

```
print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)
```

```
print(type(X_train), type(X_test), type(y_train), type(y_test))
```

```
>> torch.Size([404, 2]) torch.Size([102, 2]) torch.Size([404, 1]) torch.Size([102, 1])  
<class 'torch.Tensor'> <class 'torch.Tensor'> <class 'torch.Tensor'> <class 'torch.Tensor'>
```

```
train_dataset = MyDataset(X_train, y_train)
```

```
test_dataset  = MyDataset(X_test, y_test)
```

```
train_data_iterator = DataLoader(train_dataset, shuffle=True, batch_size=16)
```

```
test_data_iterator  = DataLoader(test_dataset, shuffle=True, batch_size=16)
```

```
print(train_data_iterator.batch_size)
```

```
print(next(iter(train_data_iterator))[0].shape)
```

```
>> 16, torch.Size([16, 2])
```

```
import torch
```

Tensors, DataLoaders

CPU vs. GPU

Autograd,
Computation graphs

torch.Module(NN)

CPU vs. GPU

```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')  
print(device)  
X_train.to(device)
```

CPU vs. GPU

```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')  
print(device)  
X_train.to(device)
```

cpu

```
tensor([[0.4997, 0.3922],  
        [0.4951, 0.4460],  
        [0.3288, 0.9323],  
        ...  
        [0.4838, 0.5953],  
        [0.4386, 0.5255]], dtype=torch.float64)
```

CPU vs. GPU

```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')  
print(device)  
X_train.to(device)
```

```
cuda:0  
tensor([[0.4997, 0.3922],  
        [0.4951, 0.4460],  
        [0.3288, 0.9323],  
        ...  
        [0.4838, 0.5953],  
        [0.4386, 0.5255]], dtype=torch.float64, device=cuda:0)
```




```
import torch
```

Tensors, DataLoaders

CPU vs. GPU

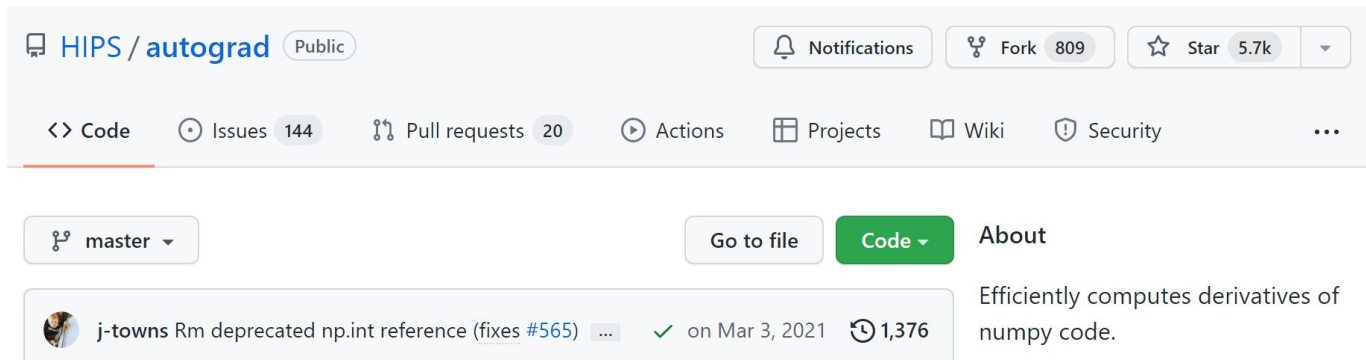
**Autograd,
Computation graphs**

torch.Module(NN)

Autograd, Computational graphs

Autograd

Pytorch's package/set of functions to perform automatic differentiation / backpropagation.



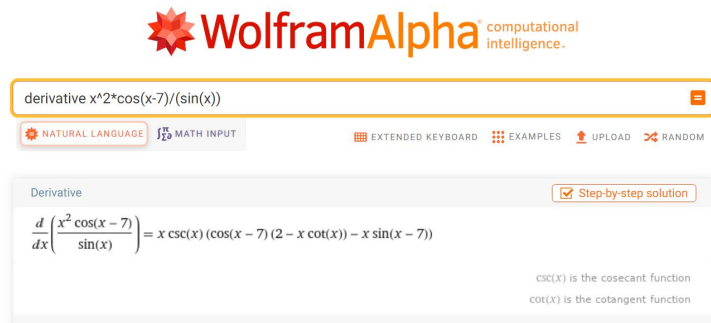
Autograd, Computational graphs

How are derivatives calculated and maintained?

Not finite differences

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}.$$

Not purely symbolic either, like Mathematica



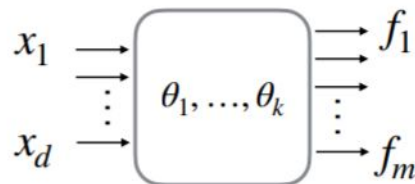
The screenshot shows the WolframAlpha website. At the top is the logo "WolframAlpha" with a red star icon and the tagline "computational intelligence". Below the logo is a search bar containing the text "derivative x^2*cos(x-7)/(sin(x))". Under the search bar are several tabs: "NATURAL LANGUAGE" (selected), "MATH INPUT", "EXTENDED KEYBOARD", "EXAMPLES", "UPLOAD", and "RANDOM". Below the search bar is a box labeled "Derivative" with a "Step-by-step solution" button. The main result is the derivative formula:
$$\frac{d}{dx} \left(\frac{x^2 \cos(x-7)}{\sin(x)} \right) = x \csc(x) (\cos(x-7) (2 - x \cot(x)) - x \sin(x-7))$$
. At the bottom right, there are two small text annotations: "csc(x) is the cosecant function" and "cot(x) is the cotangent function".

Deep learning of transformations

- We can piece together arbitrary (differentiable) adjustable transformations
- In order to train their compositions end-to-end, we need to be able to compute three things from each such transformation

- (1) how outputs are obtained as a function of the inputs and parameters (forward calc)

$$f_j = f_j(x_1, \dots, x_d; \theta_1, \dots, \theta_k), \quad j \in [m]$$



- (2) how change in the inputs change the outputs (backprop)

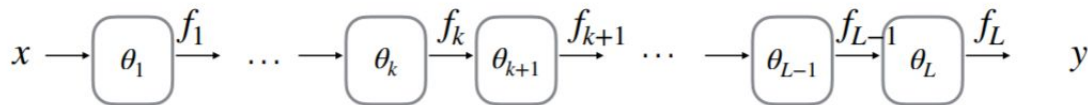
$$\frac{df_j}{dx_i}, \quad i \in [d], j \in [m]$$

- (3) how outputs change if we change the parameters (param update)

$$\frac{df_j}{d\theta_l}, \quad l \in [k], j \in [m]$$

Autograd, Computational graphs

Backpropagation



$$f_k = \tanh(\theta_k f_{k-1})$$

$$\text{Loss}(y, f_L) = \frac{1}{2}(y - f_L)^2$$

$$\theta_k \leftarrow \theta_k - \eta \frac{d\text{Loss}(y, f_L)}{d\theta_k}$$

$$\frac{d\text{Loss}(y, f_L)}{d\theta_k} = \frac{df_k}{d\theta_k} \frac{df_{k+1}}{df_k} \dots \frac{df_{L-1}}{df_{L-2}} \frac{df_L}{df_{L-1}} \frac{d\text{Loss}(y, f_L)}{df_L}$$

Autograd, Computational graphs

The screenshot shows the GitHub interface for the `pytorch/pytorch` repository. The repository is public and has 54.1k stars. The file path is `pytorch / tools / autograd / derivatives.yaml`. The file is 138 KB and contains 2600 lines of code. The commit history shows a recent commit by `bdhirsh` 8 days ago. The file is currently being viewed in the 'Raw' mode.

pytorch / pytorch Public

Notifications Fork 15k Star 54.1k

Code Issues 5k+ Pull requests 3.6k Actions Projects 20 Wiki Security

master pytorch / tools / autograd / derivatives.yaml Go to file

bdhirsh avoiding adding some functions to the public python API before 1... Latest commit f87f753 8 days ago History

166 contributors

2600 lines (2021 sloc) 138 KB Raw Blame

```
1 # Defines derivative formulas and Python signatures of methods on Variable
```

<https://github.com/pytorch/pytorch/blob/master/tools/autograd/derivatives.yaml>

Computation graph

```
a = torch.tensor(1., requires_grad = True)
```

```
b = torch.tensor(10., requires_grad = True)
```

```
w1 = torch.randn(1, requires_grad = True)
```

```
w2 = torch.randn(1, requires_grad = True)
```

```
c = w1*a + w2*b
```

```
L = 10 - c
```

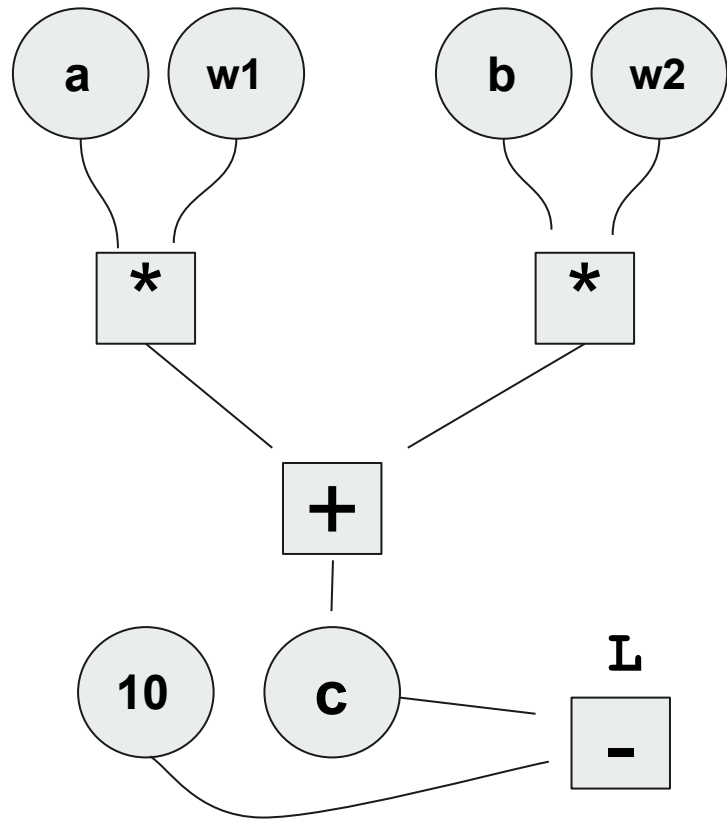
Computation graph

```
a = torch.tensor(1., requires_grad = True)
b = torch.tensor(10., requires_grad = True)
```

```
w1 = torch.randn(1, requires_grad = True)
w2 = torch.randn(1, requires_grad = True)
```

```
c = w1*a + w2*b
```

```
L = 10 - c
```



Computation graph

```
a = torch.tensor(1., requires_grad = True)
```

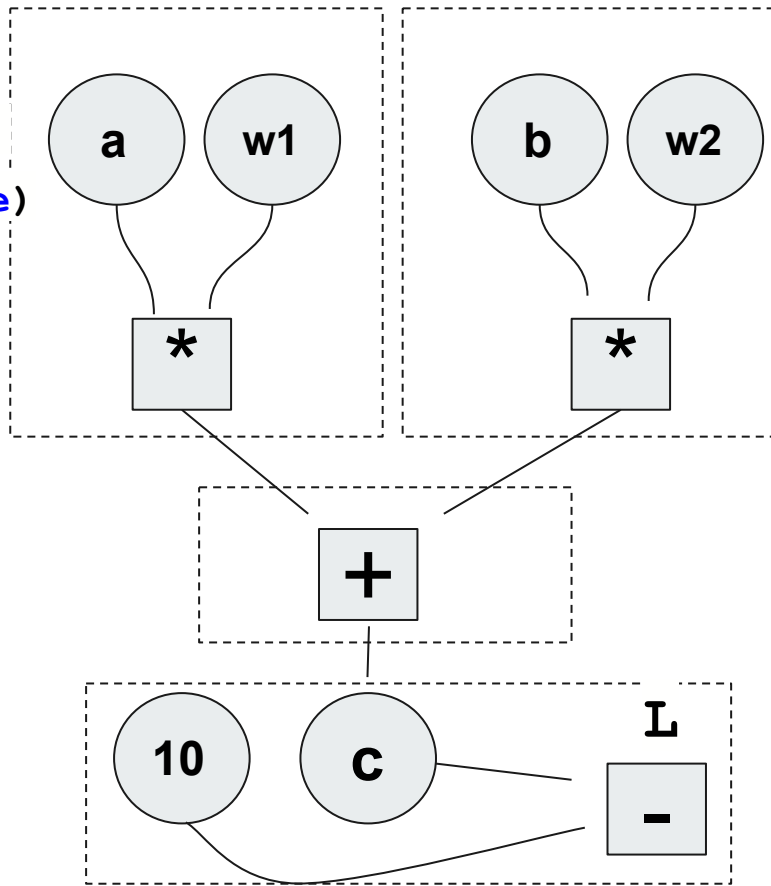
```
b = torch.tensor(10., requires_grad = True)
```

```
w1 = torch.randn(1, requires_grad = True)
```

```
w2 = torch.randn(1, requires_grad = True)
```

```
c = w1*a + w2*b
```

```
L = 10 - c
```



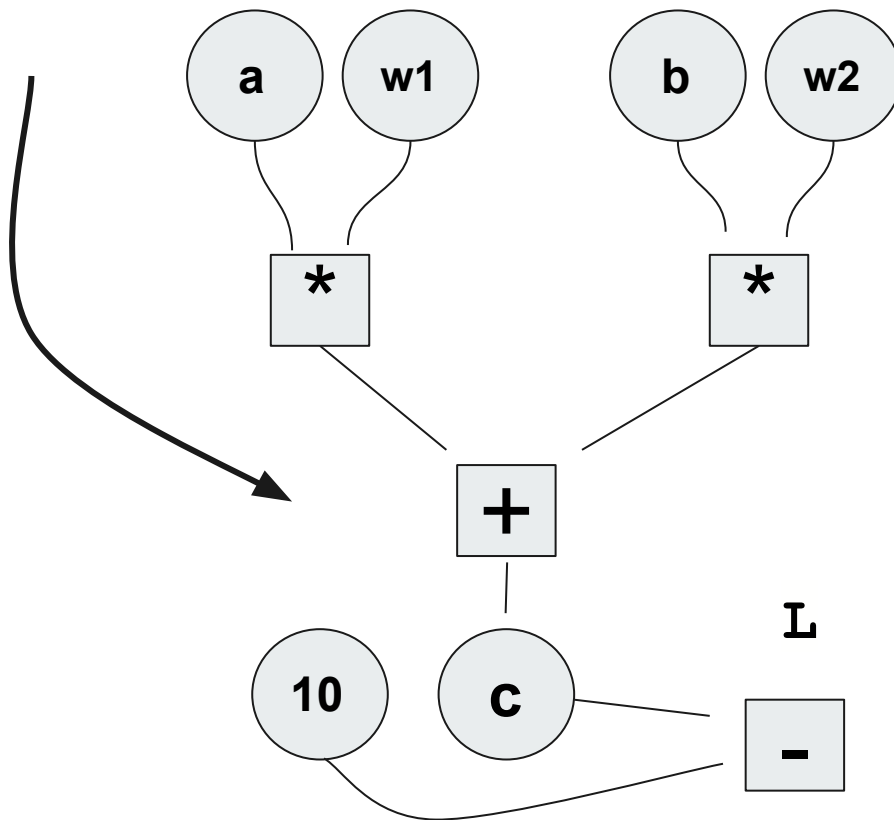
Computation graph

$$c = w1 * a + w2 * b$$

$$L = 10 - c$$

forward

$a * w1$

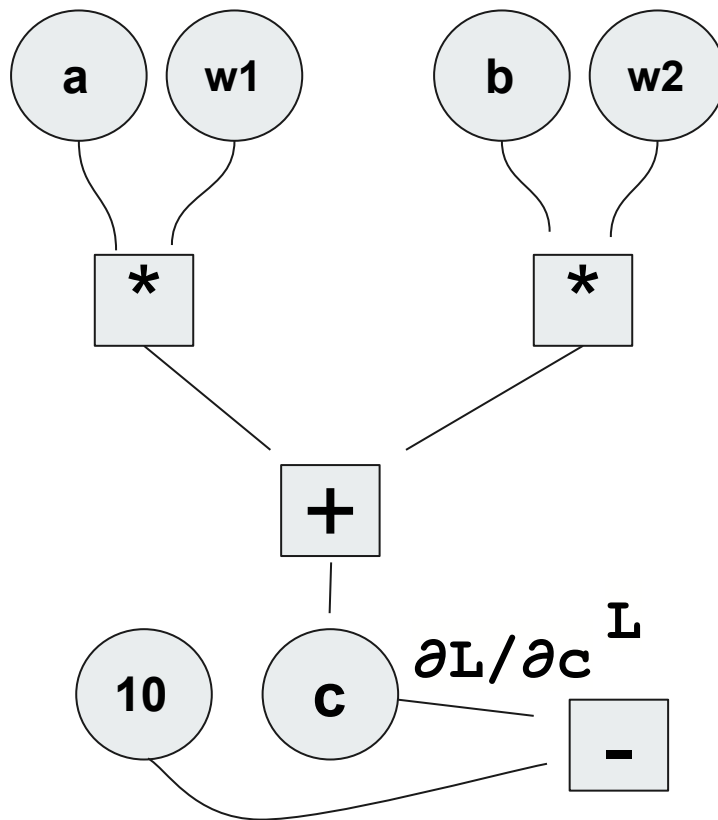


Computation graph

$$c = w1 * a + w2 * b$$

$$L = 10 - c$$

backward



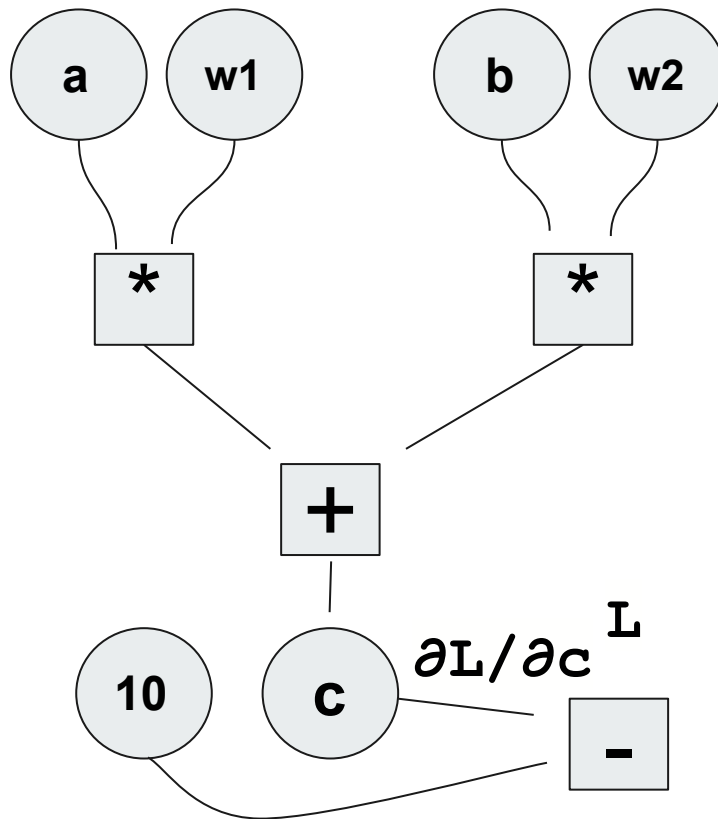
Computation graph

$$c = w1 * a + w2 * b$$

$$L = 10 - c$$

backward

$$c.\text{grad} = \partial L / \partial c$$

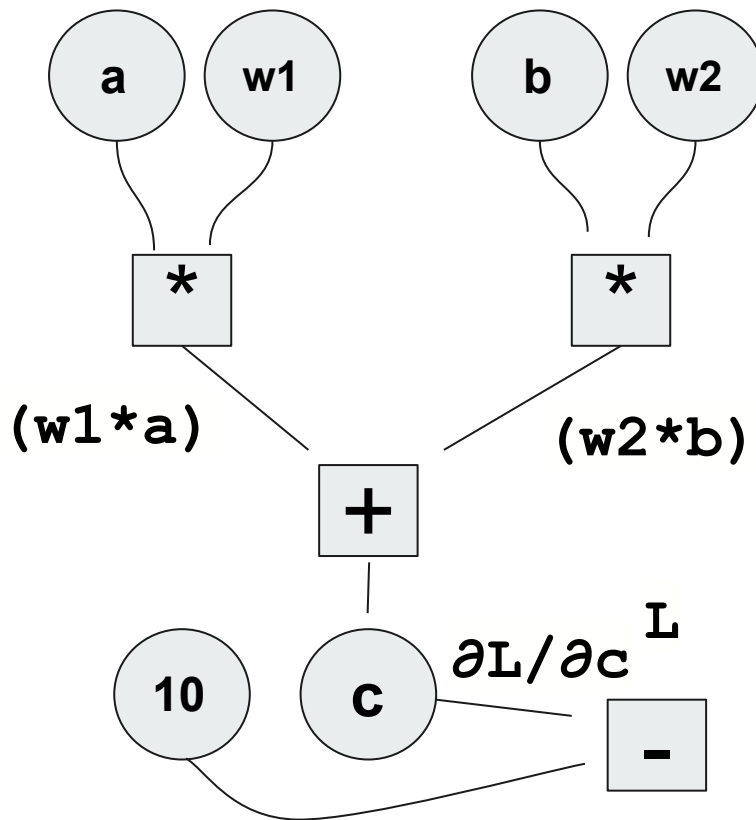


Computation graph

$$c = w1*a + w2*b$$

$$L = 10 - c$$

backward



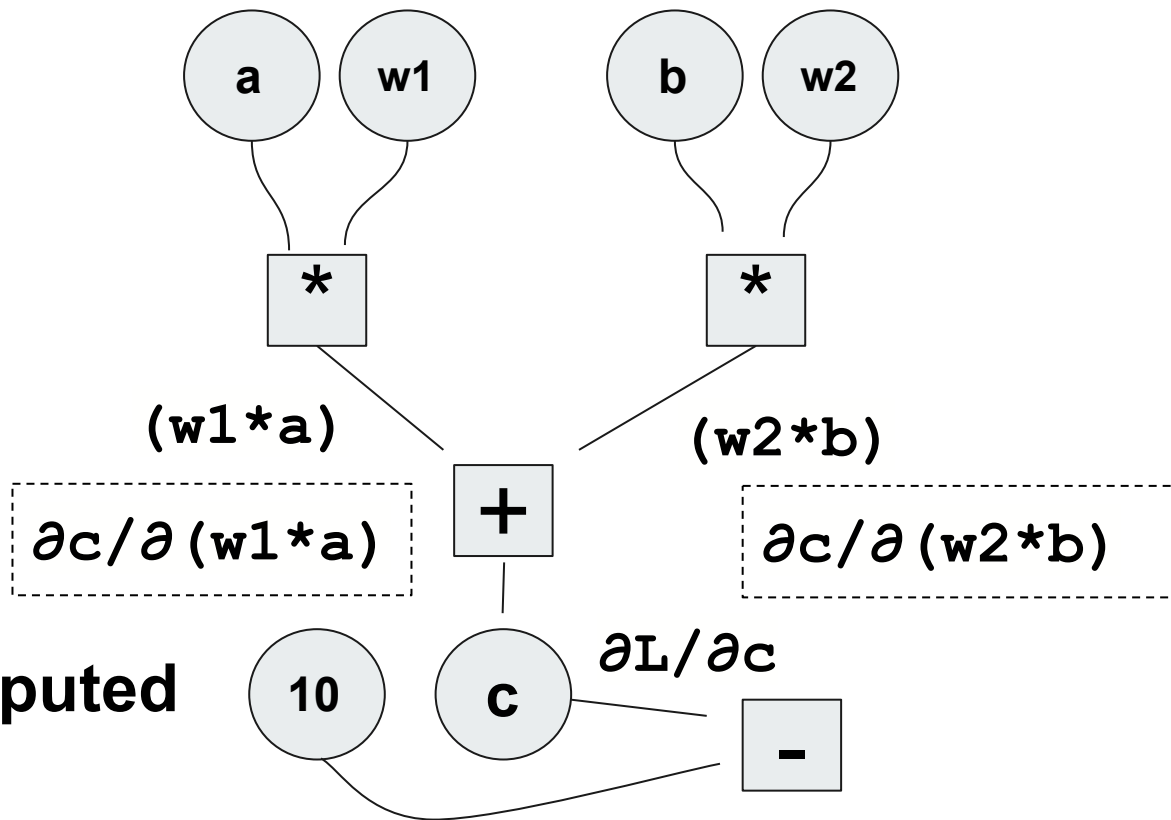
Computation graph

$$c = w1*a + w2*b$$

$$L = 10 - c$$

backward

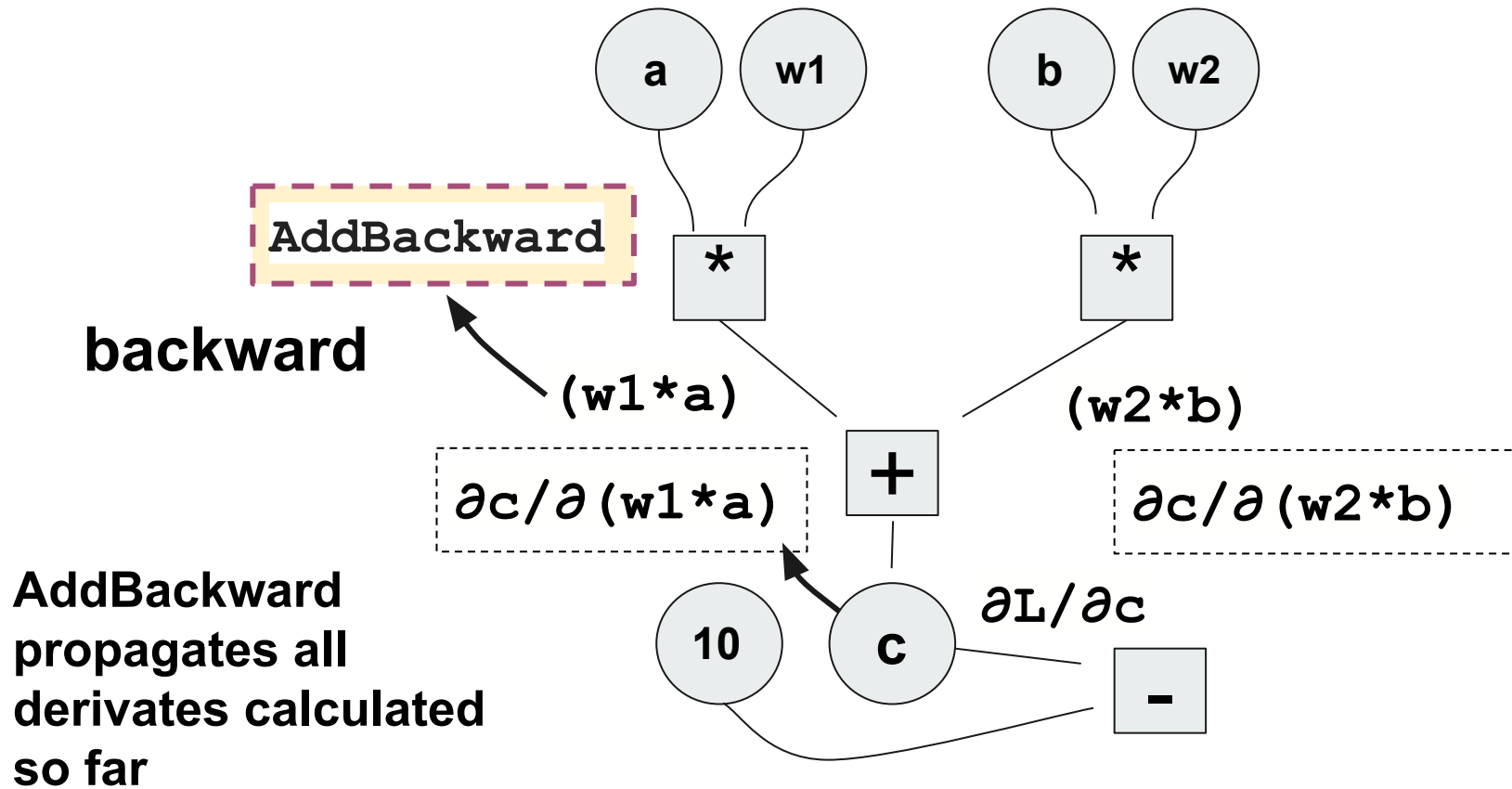
Locally computed
at c



Computation graph

$$c = w1*a + w2*b$$

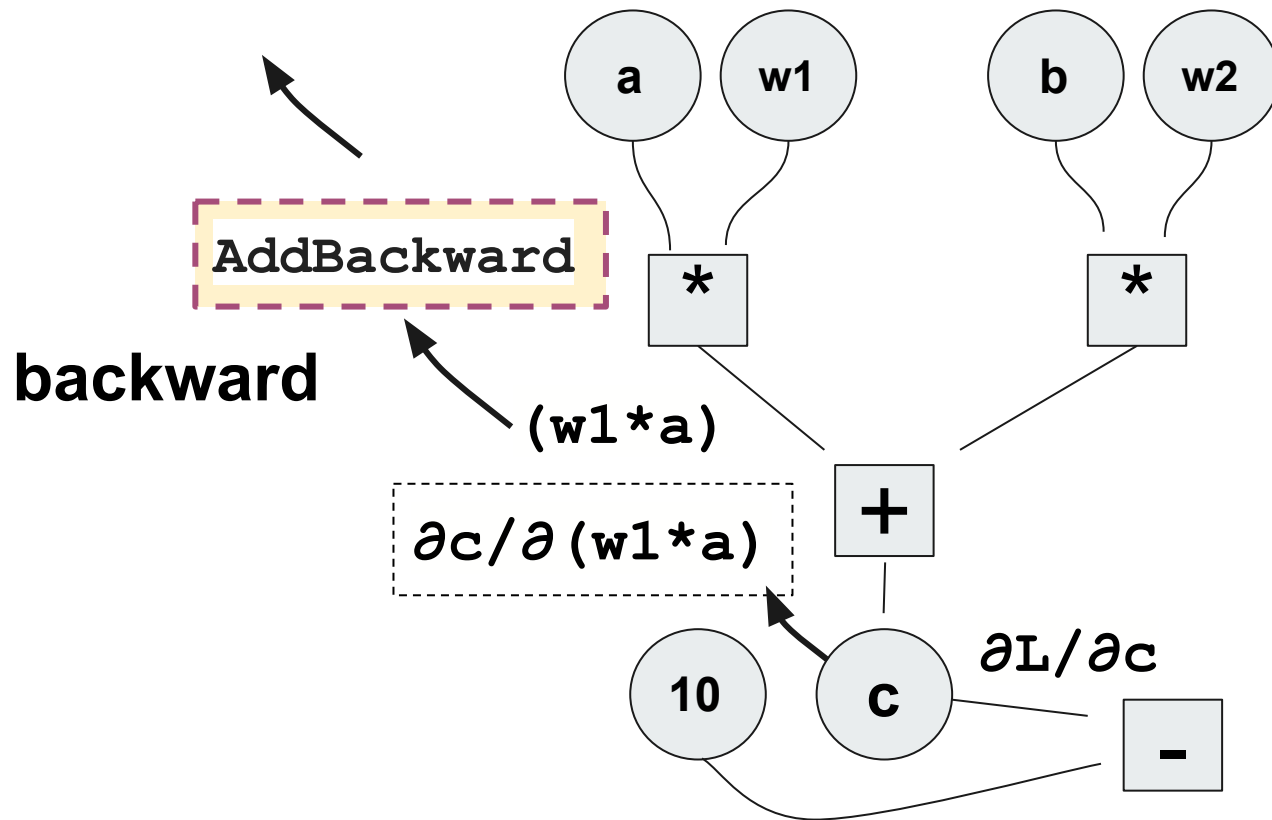
$$L = 10 - c$$



$$c = w1 * a + w2 * b$$

$$L = 10 - c$$

$$\frac{\partial c}{\partial (w1 * a)} \times \frac{\partial L}{\partial c}$$

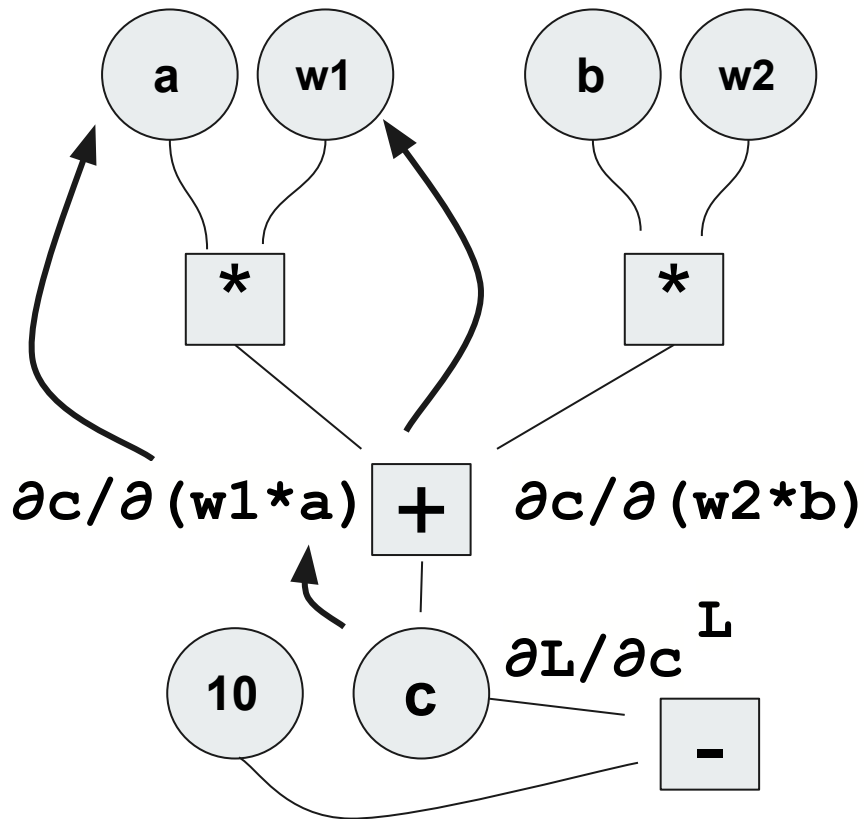


$$\partial (w1 * a) / \partial a$$

$$\partial (w1 * a) / \partial (w1)$$

$$\partial L / \partial a$$

backward



$c = w_1 * a + w_2 * b$

$L = 10 - c$

```
c.data is:  tensor([14.7042])
```

```
c.grad is:  None
```

```
c.grad_fn is:  <AddBackward0 object at 0x7f600a54f1d0>
```

```
c.is_leaf is:  False
```

```
c.requires_grad is:  True
```

```
/usr/local/lib/python3.7/dist-packages/torch/_tensor.py:1013:
```

```
UserWarning: The .grad attribute of a Tensor that is not a leaf Tensor  
is being accessed. Its .grad attribute won't be populated during  
autograd.backward(). If you indeed want the .grad field to be populated  
for a non-leaf Tensor, use .retain_grad() on the non-leaf Tensor.
```

```
c = w1*a + w2*b
```

```
L = 10 - c
```

```
c.retain_grad()
```

```
L.backward()
```

```
c.data is:  tensor([14.7042])
```

```
c.grad is:  tensor([-1.])
```

```
c.grad_fn is:  <AddBackward0 object at 0x7f600a52e590>
```

```
c.is_leaf is:  False
```

```
c.requires_grad is:  True
```

Helps you optimize variables

```
w1 = w1 - learning_rate * w1.grad
```

Helps you optimize variables

```
w1 = w1 - learning_rate * w1.grad
```

For *torch.nn* models, Pytorch has an “optimizer” which updates these parameters internally when you do an *optimizer.step*

Additional resources - Automatic differentiation

- Minitorch, by Sasha Rush

<https://minitorch.github.io/>, <https://github.com/minitorch/Module-1>

- Autodidact: a pedagogical implementation of Autograd, by Matthew Johnson

<https://github.com/mattjj/autodidact>

- Elliot Waite - Pytorch autograd explained

<https://www.youtube.com/watch?v=MswxJw-8PvE>

- Automatic differentiation in machine learning: a survey, by Baydin et al.

<https://arxiv.org/abs/1502.05767>

```
import torch
```

Tensors, DataLoaders

CPU vs. GPU

Autograd,
Computation graphs

torch.Module(NN)

Our dataset

```
print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)
print(type(X_train), type(X_test), type(y_train), type(y_test))

>> torch.Size([404, 2]) torch.Size([102, 2]) torch.Size([404, 1]) torch.Size([102, 1])
<class 'torch.Tensor'> <class 'torch.Tensor'> <class 'torch.Tensor'> <class 'torch.Tensor'>
```

```
train_dataset = MyDataset(X_train, y_train)
test_dataset  = MyDataset(X_test, y_test)
```

```
train_data_iterator = DataLoader(train_dataset, shuffle=True, batch_size=16)
test_data_iterator  = DataLoader(test_dataset, shuffle=True, batch_size=16)
```


torch.Module(NN)

```
import torch.nn
```

```
class FullyConnectedModel(nn.Module):  
    def __init__(self, input_dims=2, output_dims=1):  
        super(FullyConnectedModel, self).__init__()  
        self.fc1 = nn.Linear(input_dims, output_dims)  
  
    def forward(self, x):  
        # print(x.shape)  
        self.output = self.fc1(x)  
        return self.output
```

Training

```
model_fc = FullyConnectedModel()
model_fc.to(device)
optimizer = optim.AdamW(model_fc.parameters(), lr = 15, eps = 1e-5)
epochs = 15
loss_function = nn.MSELoss()
total_iters = len(train_data_iterator) * epochs
scheduler = optim.lr_scheduler.LinearLR(optimizer, total_iters=total_iters)

for epoch in tqdm(range(epochs)):
    # Perform one full pass over the training set.
    train_true, train_predict, train_loss = train(model_fc,
                                                    train_data_iterator,
                                                    loss_function,
                                                    optimizer,
                                                    scheduler,
                                                    device)

    train_mse = mean_squared_error(train_true, train_predict)
```

Training

```
model_fc = FullyConnectedModel()
model_fc.to(device)
optimizer = optim.AdamW(model_fc.parameters(), lr = 1e-5, eps = 1e-5)
epochs = 15
loss_function = nn.MSELoss()
total_iters = len(train_data_iterator) * epochs
scheduler = optim.lr_scheduler.LinearLR(optimizer, total_iters=total_iters)

for epoch in tqdm(range(epochs)):
    # Perform one full pass over the training set.
    train_true, train_predict, train_loss = train(model_fc,
                                                    train_data_iterator,
                                                    loss_function,
                                                    optimizer,
                                                    scheduler,
                                                    device)
    train_mse = mean_squared_error(train_true, train_predict)
```

Training

```
model_fc = FullyConnectedModel()
model_fc.to(device)

optimizer = optim.AdamW(model_fc.parameters(), lr = 15, eps = 1e-5)
epochs = 15

loss_function = nn.MSELoss()

total_iters = len(train_data_iterator) * epochs

scheduler = optim.lr_scheduler.LinearLR(optimizer, total_iters=total_iters)

for epoch in tqdm(range(epochs)):
    # Perform one full pass over the training set.
    train_true, train_predict, train_loss = train(model_fc,
                                                    train_data_iterator,
                                                    loss_function,
                                                    optimizer,
                                                    scheduler,
                                                    device)

    train_mse = mean_squared_error(train_true, train_predict)
```

Training

```
model_fc = FullyConnectedModel()
model_fc.to(device)
optimizer = optim.AdamW(model_fc.parameters(), lr = 1e-5, eps = 1e-5)
epochs = 15
loss_function = nn.MSELoss()
total_iters = len(train_data_iterator) * epochs
scheduler = optim.lr_scheduler.LinearLR(optimizer, total_iters=total_iters)

for epoch in tqdm(range(epochs)):
    # Perform one full pass over the training set.
    train_true, train_predict, train_loss = train(model_fc,
                                                    train_data_iterator,
                                                    loss_function,
                                                    optimizer,
                                                    scheduler,
                                                    device)

    train_mse = mean_squared_error(train_true, train_predict)
```

Training

```
model_fc = FullyConnectedModel()
model_fc.to(device)
optimizer = optim.AdamW(model_fc.parameters(), lr = 15, eps = 1e-5)
epochs = 15
loss_function = nn.MSELoss()
total_iters = len(train_data_iterator) * epochs
scheduler = optim.lr_scheduler.LinearLR(optimizer, total_iters=total_iters)

for epoch in tqdm(range(epochs)):
    # Perform one full pass over the training set.
    train_true, train_predict, train_loss = train(model_fc,
                                                    train_data_iterator,
                                                    loss_function,
                                                    optimizer,
                                                    scheduler,
                                                    device)

    train_mse = mean_squared_error(train_true, train_predict)
```

Training

```
model_fc = FullyConnectedModel()
model_fc.to(device)
optimizer = optim.AdamW(model_fc.parameters(), lr = 15, eps = 1e-5)
epochs = 15
loss_function = nn.MSELoss()
total_iters = len(train_data_iterator) * epochs
scheduler = optim.lr_scheduler.LinearLR(optimizer, total_iters=total_iters)
```

```
for epoch in tqdm(range(epochs)):
    # Perform one full pass over the training set.
    train_true, train_predict, train_loss = train(model_fc,
                                                    train_data_iterator,
                                                    loss_function,
                                                    optimizer,
                                                    scheduler,
                                                    device)

    train_mse = mean_squared_error(train_true, train_predict)
```

torch.Module(NN)

```
def train(model_fc, dataloader, loss_fn_, optimizer_, scheduler_, device_):  
    # Put the model into training mode.  
    model_fc.train()  
  
    # For each batch of training data...  
    # Each batch has (x, y) pairs  
    for batch in tqdm(dataloader, total=len(dataloader)):  
        <DO SOMETHING IN EACH BATCH>  
  
    # Calculate the average loss over the training data.  
    avg_epoch_loss = total_loss / len(dataloader)  
  
    return y_true, y_predicted, avg_epoch_loss
```


Do something in each batch

```
y_true += batch[1].cpu().numpy().flatten().tolist()
```

```
# move batch to device
```

```
model_fc.zero_grad()
```

```
outputs = model_fc(batch[0]).squeeze(1)
```

```
loss = loss_fn_(outputs, batch[1].squeeze(1))
```

```
total_loss += loss.item()
```

```
outputs = outputs.detach().cpu().numpy().flatten().tolist()
```

```
y_predicted += outputs
```

Do something in each batch

```
y_true += batch[1].cpu().numpy().flatten().tolist()
```

```
# move batch to device
```

```
model_fc.zero_grad()
```

```
outputs = model_fc(batch[0]).squeeze(1)
```

```
loss = loss_fn_(outputs, batch[1].squeeze(1))
```

```
total_loss += loss.item()
```

```
outputs = outputs.detach().cpu().numpy().flatten().tolist()
```

```
y_predicted += outputs
```

Do something in each batch

- a `optimizer_.step()`
- b `scheduler_.step()`
- c `loss.backward()`

```
y_true += batch[1].cpu().numpy().flatten().tolist()
```

```
# move batch to device
```

```
model_fc.zero_grad()
```

```
outputs = model_fc(batch[0]).squeeze(1)
```

```
loss = loss_fn_(outputs, batch[1].squeeze(1))
```

```
total_loss += loss.item()
```

1

2

```
outputs = outputs.detach().cpu().numpy().flatten().tolist()
```

```
y_predicted += outputs
```

Do something in each batch

```
y_true += batch[1].cpu().numpy().flatten().tolist()
```

```
# move batch to device
```

```
model_fc.zero_grad()
```

```
outputs = model_fc(batch[0]).squeeze(1)
```

```
loss = loss_fn_(outputs, batch[1].squeeze(1))
```

```
total_loss += loss.item()
```

```
loss.backward()
```

```
optimizer_.step()
```

```
scheduler_.step()
```

```
outputs = outputs.detach().cpu().numpy().flatten().tolist()
```

```
y_predicted += outputs
```

Do something in each batch

```
y_true += batch[1].cpu().numpy().flatten().tolist()
```

```
# move batch to device
```

```
model_fc.zero_grad()
```

```
outputs = model_fc(batch[0]).squeeze(1)
```

```
loss = loss_fn_(outputs, batch[1].squeeze(1))
```

```
total_loss += loss.item()
```

```
loss.backward()
```

```
optimizer_.step()
```

```
scheduler_.step()
```

```
outputs = outputs.detach().cpu().numpy().flatten().tolist()
```

```
y_predicted += outputs
```

Do something in each batch

```
y_true += batch[1].cpu().numpy().flatten().tolist()
```

```
# move batch to device
```

```
model_fc.zero_grad()
```

```
outputs = model_fc(batch[0]).squeeze(1)
```

```
loss = loss_fn_(outputs, batch[1].squeeze(1))
```

```
total_loss += loss.item()
```

```
loss.backward()
```

```
optimizer_.step()
```

```
scheduler_.step()
```

```
outputs = outputs.detach().cpu().numpy().flatten().tolist()
```

```
y_predicted += outputs
```

Debugging

Trivial – print, log, pdb

```
print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)
```

```
print(type(X_train), type(X_test), type(y_train), type(y_test))
```

```
>> torch.Size([404, 2]) torch.Size([102, 2]) torch.Size([404, 1]) torch.Size([102, 1])  
<class 'torch.Tensor'> <class 'torch.Tensor'> <class 'torch.Tensor'> <class 'torch.Tensor'>
```

Installation, setting up GPU, other troubleshooting

<https://pytorch.org/tutorials/>

<https://discuss.pytorch.org/>

<https://pytorch.org/tutorials/beginner/basics/intro.html>

https://pytorch.org/tutorials/recipes/recipes_index.html

Pytorch in other domains

<https://pytorch.org/ecosystem/>

AllenNLP

Allen AI's NLP library

FastAI

Fast AI

Pytorch Lightning

Other utilities

Pyro

Probabilistic programming



TensorFlow



Tensorflow Overview

- Tensorflow V2 From Scratch
- Keras API
- Other Ecosystem Packages: JAX

TF V2: Linear Regression From Scratch

Variable Definitions

class **LinearModel**:

def **__init__**(self, learning_rate=0.1):

self.w = tf.Variable(np.random.randn(), name="W")

self.bias = tf.Variable(np.random.randn(), name="b")

self.learning_rate = learning_rate

self.optimizer_sgd = tf.optimizers.SGD(self.learning_rate) # sgd optimizer.

self.optimizer_adam = tf.optimizers.Adam() # adam optimizer.

self.training_steps = 1000 # number of training steps.

self.log_train = 100 # option to print out loss as you train your model.

def **__call__**(self, x):

return self.w*x + self.bias

def **predict**(self, x):

return self.w*x + self.bias



def **loss**(self, y, y_pred):

return tf.reduce_mean(tf.square(y - y_pred))

def **train**(self, x, y):

for step in range(1, self.training_steps + 1):

with **tf.GradientTape()** as tape:

predictions = self.w*x + self.bias

current_loss = self.loss(y, predictions)

delta_w, delta_bias = tape.gradient(current_loss, [self.w, self.bias])

self.optimizer_sgd.apply_gradients(zip([delta_w, delta_bias], [self.w, self.bias]))

TF V2: Linear Regression From Scratch

class **LinearModel**:

def **__init__**(self, learning_rate=0.1):

self.w = tf.Variable(np.random.randn(), name="W")

self.bias = tf.Variable(np.random.randn(), name="b")

self.learning_rate = learning_rate

self.optimizer_sgd = tf.optimizers.SGD(self.learning_rate) # sgd optimizer.

self.optimizer_adam = tf.optimizers.Adam() # adam optimizer.

self.training_steps = 1000 # number of training steps.

self.log_train = 100 # option to print out loss as you train your model.

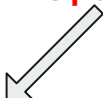
def **__call__**(self, x):

return self.w*x + self.bias

def **predict**(self, x):

return self.w*x + self.bias

**SGD
Optimizer**



def **loss**(self, y, y_pred):

return tf.reduce_mean(tf.square(y - y_pred))

def **train**(self, x, y):

for step in range(1, self.training_steps + 1):

with **tf.GradientTape()** as tape:

predictions = self.w*x + self.bias

current_loss = self.loss(y, predictions)

delta_w, delta_bias = tape.gradient(current_loss, [self.w, self.bias])

self.optimizer_sgd.apply_gradients(zip([delta_w, delta_bias], [self.w, self.bias]))

TF V2: Linear Regression From Scratch

class **LinearModel**:

```
def __init__(self, learning_rate=0.1):
    self.w = tf.Variable(np.random.randn(), name="W")
    self.bias = tf.Variable(np.random.randn(), name="b")
    self.learning_rate = learning_rate
    self.optimizer_sgd = tf.optimizers.SGD(self.learning_rate) # sgd optimizer.
    self.optimizer_adam = tf.optimizers.Adam() # adam optimizer.
    self.training_steps = 1000 # number of training steps.
    self.log_train = 100 # option to print out loss as you train your model.

def __call__(self, x):
    return self.w*x + self.bias

def predict(self, x):
    return self.w*x + self.bias
```

Training Iterations



```
def loss(self, y, y_pred):
    return tf.reduce_mean(tf.square(y - y_pred))

def train(self, x, y):
    for step in range(1, self.training_steps + 1):
        with tf.GradientTape() as tape:
            predictions = self.w*x + self.bias
            current_loss = self.loss(y, predictions)
            delta_w, delta_bias = tape.gradient(current_loss, [self.w, self.bias])
            self.optimizer_sgd.apply_gradients(zip([delta_w, delta_bias], [self.w, self.bias]))
```

TF V2: Linear Regression From Scratch

Mean Squared Error

class **LinearModel**:

```
def __init__(self, learning_rate=0.1):
    self.w = tf.Variable(np.random.randn(), name="W")
    self.bias = tf.Variable(np.random.randn(), name="b")
    self.learning_rate = learning_rate
    self.optimizer_sgd = tf.optimizers.SGD(self.learning_rate) # sgd optimizer.
    self.optimizer_adam = tf.optimizers.Adam() # adam optimizer.
    self.training_steps = 1000 # number of training steps.
    self.log_train = 100 # option to print out loss as you train your model.

def __call__(self, x):
    return self.w*x + self.bias

def predict(self, x):
    return self.w*x + self.bias
```

```
def loss(self, y, y_pred):
    return tf.reduce_mean(tf.square(y - y_pred))
```

```
def train(self, x, y):
    for step in range(1, self.training_steps + 1):
        with tf.GradientTape() as tape:
            predictions = self.w*x + self.bias
            current_loss = self.loss(y, predictions)
            delta_w, delta_bias = tape.gradient(current_loss, [self.w, self.bias])
            self.optimizer_sgd.apply_gradients(zip([delta_w, delta_bias], [self.w, self.bias]))
```

TF V2: Linear Regression From Scratch

class **LinearModel**:

```
def __init__(self, learning_rate=0.1):
    self.w = tf.Variable(np.random.randn(), name="W")
    self.bias = tf.Variable(np.random.randn(), name="b")
    self.learning_rate = learning_rate
    self.optimizer_sgd = tf.optimizers.SGD(self.learning_rate) # sgd optimizer.
    self.optimizer_adam = tf.optimizers.Adam() # adam optimizer.
    self.training_steps = 1000 # number of training steps.
    self.log_train = 100 # option to print out loss as you train your model.

def __call__(self, x):
    return self.w*x + self.bias

def predict(self, x):
    return self.w*x + self.bias
```

```
def loss(self, y, y_pred):
    return tf.reduce_mean(tf.square(y - y_pred))
```

Training Loop



```
def train(self, x, y):
    for step in range(1, self.training_steps + 1):
        with tf.GradientTape() as tape:
            predictions = self.w*x + self.bias
            current_loss = self.loss(y, predictions)
            delta_w, delta_bias = tape.gradient(current_loss, [self.w, self.bias])
            self.optimizer_sgd.apply_gradients(zip([delta_w, delta_bias], [self.w, self.bias]))
```


TF V2: Linear Regression From Scratch

class **LinearModel**:

```
def __init__(self, learning_rate=0.1):
    self.w = tf.Variable(np.random.randn(), name="W")
    self.bias = tf.Variable(np.random.randn(), name="b")
    self.learning_rate = learning_rate
    self.optimizer_sgd = tf.optimizers.SGD(self.learning_rate) # sgd optimizer.
    self.optimizer_adam = tf.optimizers.Adam() # adam optimizer.
    self.training_steps = 1000 # number of training steps.
    self.log_train = 100 # option to print out loss as you train your model.

def __call__(self, x):
    return self.w*x + self.bias

def predict(self, x):
    return self.w*x + self.bias
```

```
def loss(self, y, y_pred):
    return tf.reduce_mean(tf.square(y - y_pred))
```

```
def train(self, x, y):
```

```
    for step in range(1, self.training_steps + 1):
```

```
        with tf.GradientTape() as tape:
```

```
            predictions = self.w*x + self.bias
```

```
            current_loss = self.loss(y, predictions)
```

```
            delta_w, delta_bias = tape.gradient(current_loss, [self.w, self.bias])
```

```
            self.optimizer_sgd.apply_gradients(zip([delta_w, delta_bias], [self.w, self.bias]))
```

For each iteration



TF V2: Linear Regression From Scratch

class **LinearModel**:

```
def __init__(self, learning_rate=0.1):
    self.w = tf.Variable(np.random.randn(), name="W")
    self.bias = tf.Variable(np.random.randn(), name="b")
    self.learning_rate = learning_rate
    self.optimizer_sgd = tf.optimizers.SGD(self.learning_rate) # sgd optimizer.
    self.optimizer_adam = tf.optimizers.Adam() # adam optimizer.
    self.training_steps = 1000 # number of training steps.
    self.log_train = 100 # option to print out loss as you train your model.

def __call__(self, x):
    return self.w*x + self.bias

def predict(self, x):
    return self.w*x + self.bias
```

```
def loss(self, y, y_pred):
    return tf.reduce_mean(tf.square(y - y_pred))
```

```
def train(self, x, y):
    for step in range(1, self.training_steps + 1):
        with tf.GradientTape() as tape:
            predictions = self.w*x + self.bias
            current_loss = self.loss(y, predictions)
            delta_w, delta_bias = tape.gradient(current_loss, [self.w, self.bias])
            self.optimizer_sgd.apply_gradients(zip([delta_w, delta_bias], [self.w, self.bias]))
```

Key Autograd Primitive



TF V2: Linear Regression From Scratch

class **LinearModel**:

```
def __init__(self, learning_rate=0.1):
    self.w = tf.Variable(np.random.randn(), name="W")
    self.bias = tf.Variable(np.random.randn(), name="b")
    self.learning_rate = learning_rate
    self.optimizer_sgd = tf.optimizers.SGD(self.learning_rate) # sgd optimizer.
    self.optimizer_adam = tf.optimizers.Adam() # adam optimizer.
    self.training_steps = 1000 # number of training steps.
    self.log_train = 100 # option to print out loss as you train your model.

def __call__(self, x):
    return self.w*x + self.bias

def predict(self, x):
    return self.w*x + self.bias
```

```
def loss(self, y, y_pred):
    return tf.reduce_mean(tf.square(y - y_pred))
```

```
def train(self, x, y):
    for step in range(1, self.training_steps + 1):
        with tf.GradientTape() as tape:
            predictions = self.w*x + self.bias
            current_loss = self.loss(y, predictions)
            delta_w, delta_bias = tape.gradient(current_loss, [self.w, self.bias])
            self.optimizer_sgd.apply_gradients(zip([delta_w, delta_bias], [self.w, self.bias]))
```

Get Current Predictions



TF V2: Linear Regression From Scratch

class **LinearModel**:

```
def __init__(self, learning_rate=0.1):
    self.w = tf.Variable(np.random.randn(), name="W")
    self.bias = tf.Variable(np.random.randn(), name="b")
    self.learning_rate = learning_rate
    self.optimizer_sgd = tf.optimizers.SGD(self.learning_rate) # sgd optimizer.
    self.optimizer_adam = tf.optimizers.Adam() # adam optimizer.
    self.training_steps = 1000 # number of training steps.
    self.log_train = 100 # option to print out loss as you train your model.

def __call__(self, x):
    return self.w*x + self.bias

def predict(self, x):
    return self.w*x + self.bias
```

```
def loss(self, y, y_pred):
    return tf.reduce_mean(tf.square(y - y_pred))
```

```
def train(self, x, y):
```

```
    for step in range(1, self.training_steps + 1):
```

```
        with tf.GradientTape() as tape:
```

```
            predictions = self.w*x + self.bias
```

```
            current_loss = self.loss(y, predictions)
```

```
            delta_w, delta_bias = tape.gradient(current_loss, [self.w, self.bias])
```

```
            self.optimizer_sgd.apply_gradients(zip([delta_w, delta_bias], [self.w, self.bias]))
```

Compute Loss Given Predictions



TF V2: Linear Regression From Scratch

class **LinearModel**:

```
def __init__(self, learning_rate=0.1):
    self.w = tf.Variable(np.random.randn(), name="W")
    self.bias = tf.Variable(np.random.randn(), name="b")
    self.learning_rate = learning_rate
    self.optimizer_sgd = tf.optimizers.SGD(self.learning_rate) # sgd optimizer.
    self.optimizer_adam = tf.optimizers.Adam() # adam optimizer.
    self.training_steps = 1000 # number of training steps.
    self.log_train = 100 # option to print out loss as you train your model.

def __call__(self, x):
    return self.w*x + self.bias

def predict(self, x):
    return self.w*x + self.bias
```

```
def loss(self, y, y_pred):
    return tf.reduce_mean(tf.square(y - y_pred))
```

```
def train(self, x, y):
    for step in range(1, self.training_steps + 1):
```

with **tf.GradientTape()** as tape:

```
    predictions = self.w*x + self.bias
    current_loss = self.loss(y, predictions)
```

```
    delta_w, delta_bias = tape.gradient(current_loss, [self.w, self.bias])
```

```
    self.optimizer_sgd.apply_gradients(zip([delta_w, delta_bias], [self.w, self.bias]))
```

Gradient of loss w.r.t. params



TF V2: Linear Regression From Scratch

class **LinearModel**:

```
def __init__(self, learning_rate=0.1):
    self.w = tf.Variable(np.random.randn(), name="W")
    self.bias = tf.Variable(np.random.randn(), name="b")
    self.learning_rate = learning_rate
    self.optimizer_sgd = tf.optimizers.SGD(self.learning_rate) # sgd optimizer.
    self.optimizer_adam = tf.optimizers.Adam() # adam optimizer.
    self.training_steps = 1000 # number of training steps.
    self.log_train = 100 # option to print out loss as you train your model.

def __call__(self, x):
    return self.w*x + self.bias

def predict(self, x):
    return self.w*x + self.bias
```

```
def loss(self, y, y_pred):
    return tf.reduce_mean(tf.square(y - y_pred))
```

```
def train(self, x, y):
    for step in range(1, self.training_steps + 1):
```

```
        with tf.GradientTape() as tape:
```

```
            predictions = self.w*x + self.bias
```

```
            current_loss = self.loss(y, predictions)
```

```
            delta_w, delta_bias = tape.gradient(current_loss, [self.w, self.bias])
```

```
            self.optimizer_sgd.apply_gradients(zip([delta_w, delta_bias], [self.w, self.bias]))
```

Update current params





TF V2: Linear Regression From Scratch

instantiate model

```
reg_model = LinearModel()
```

Setup Model

train model

```
reg_model.train(X_train, y_train)
```

get test predictions

```
test_predictions = reg_model(X_test)
```

```
print(reg_model.loss(test_predictions, y_test).numpy())
```



TF V2: Linear Regression From Scratch

```
# instantiate model
```

```
reg_model = LinearModel()
```

```
# train model
```

```
reg_model.train(X_train, y_train)
```

Train Model

```
# get test predictions
```

```
test_predictions = reg_model(X_test)
```

```
print(reg_model.loss(test_predictions, y_test).numpy())
```




TF V2: Linear Regression From Scratch

instantiate model

```
reg_model = LinearModel()
```

train model

```
reg_model.train(X_train, y_train)
```

get test predictions

```
test_predictions = reg_model(X_test)
```

```
print(reg_model.loss(test_predictions, y_test).numpy())
```

Test Predictions



Keras API

`torch.nn` approximately equivalent to `tf.keras.sequential`

Model Definition

```
model = tf.keras.Sequential()  
model.add(tf.keras.layers.Dense(1, input_shape=(input_dims,)))
```



Keras API

`torch.nn` approximately equivalent to `tf.keras.sequential`

Model Definition

```
model = tf.keras.Sequential()  
model.add(tf.keras.layers.Dense(1, input_shape=(input_dims,)))
```

Compile Model with Specific Loss and Optimizer

```
model.compile(optimizer='sgd', loss='mse')
```



Keras API

`torch.nn` approximately equivalent to `tf.keras.sequential`

Model Definition

```
model = tf.keras.Sequential()  
model.add(tf.keras.layers.Dense(1, input_shape=(input_dims,)))
```

Compile Model with Specific Loss and Optimizer

```
model.compile(optimizer='sgd', loss='mse')
```

Training

```
# Fit the model  
model.fit(x, y, batch_size=32, epochs=10)
```



JAX

Numpy + XLA = JAX

```
import jax.numpy as jnp
from jax import grad, jit, vmap

def predict(params, inputs):
    for W, b in params:
        outputs = jnp.dot(inputs, W) + b
        inputs = jnp.tanh(outputs) # inputs to the next layer
    return outputs                # no activation on last layer

def loss(params, inputs, targets):
    preds = predict(params, inputs)
    return jnp.sum((preds - targets)**2)

grad_loss = jit(grad(loss)) # compiled gradient evaluation function
perex_grads = jit(vmap(grad_loss, in_axes=(None, 0, 0))) # fast per-example grads
```

JIT: Just in time compilation

Vmap: Vectorized mapping

Pmap: Parallel computation



JAX: Advanced Autodiff

- Implicit Differentiation.
- Forward Mode & Reverse Mode Autodiff.
- Hessian-Vector Products or Fancy operations on the Jacobian.

```
X_tr, y_tr = load_data()

def f(x, theta): # objective function
    residual = jnp.dot(X_tr, x) - y_tr
    return (jnp.sum(residual ** 2) + theta * jnp.sum(x ** 2)) / 2

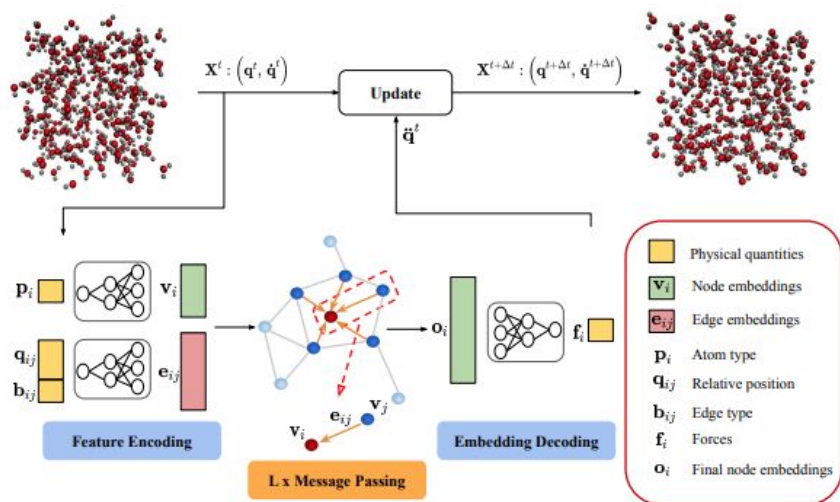
F = jax.grad(f) # optimality condition

@custom_root(F)
def ridge_solver(init_x, theta):
    del init_x # Initialization not used in this solver
    XX = jnp.dot(X_tr.T, X_tr)
    Xy = jnp.dot(X_tr.T, y_tr)
    I = jnp.eye(X_tr.shape[1])
    return jnp.linalg.solve(XX + theta * I, Xy)

init_x = None
print(jax.jacobian(ridge_solver, argnums=1)(init_x, 10.0))
```

JAX_MD

Differentiable physics simulations with a focus on molecular dynamics.



Simulation using a Graph Neural Network (See: [Li et. al, 2021](#)).



Resources

- Tensorflow V2: <https://www.tensorflow.org/tutorials/quickstart/beginner>
- Keras API Overview: https://www.tensorflow.org/api_docs/python/tf/keras/Model
- JAX: <https://jax.readthedocs.io/en/latest/>
- JAX_MD: <https://github.com/google/jax-md>